

unbound

GRACE HOPPER
CELEBRATION 2025

Model Context Protocol (MCP): The Next Frontier of Generative AI

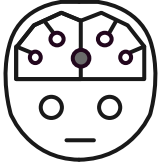
Neelam Koshiya(She/Her)
Principal Applied AI Architect, AWS

Nafi Diallo(She/Her)
Sr. Applied AI Architect, AWS

Agenda

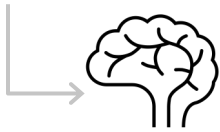
- MCP Origin & Overview
- Demo
- Building MCP Clients
- Building MCP Servers
- Industry Trends
- Workshop
- Q&A

MCP Origin & Overview



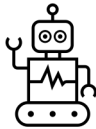
Artificial intelligence (AI)

A field of computer science that creates systems capable of performing tasks requiring human-like reasoning through algorithms, decision trees, and statistical learning methods.



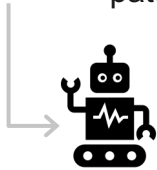
Machine learning (ML)

A branch of AI where algorithms analyze data to identify patterns and construct decision-making models with minimal human intervention.



Generative AI

A specialized branch of deep learning that leverages large-scale foundation models to create novel content and insights by learning and extrapolating patterns from massive training datasets.



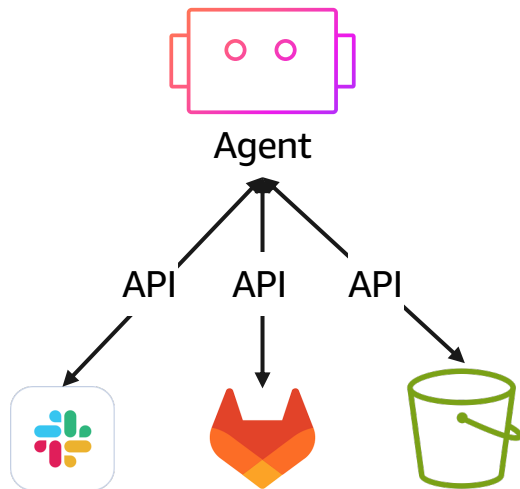
Agentic AI

Agentic AI is an autonomous system that combines large language models with goal-directed capabilities to independently plan, reason, and execute complex multi-step tasks while utilizing tools, maintaining memory, and operating within defined safety parameters to achieve specific objectives

Model Context Protocol (MCP)

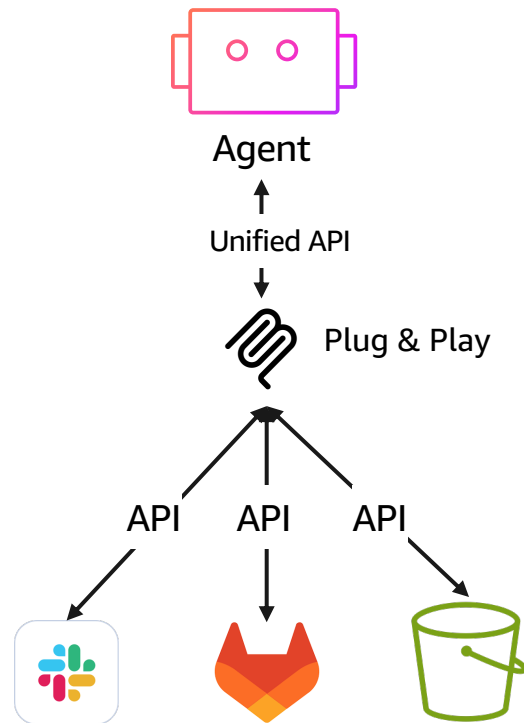
Open Standard introduced by **Anthropic** in **Nov 2024**. MCP is an open protocol that enables seamless integration between **AI apps & agents** and your **tools & data sources**.

Before MCP



M×N different integrations

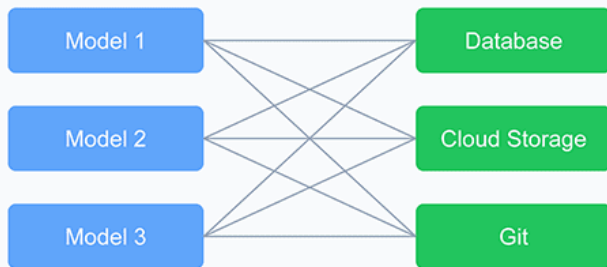
After MCP



M+N

Traditional Integration vs MCP Approach

Traditional: $N \times M$ Connections



Each model needs custom integration
with each data source

9 Total Connections

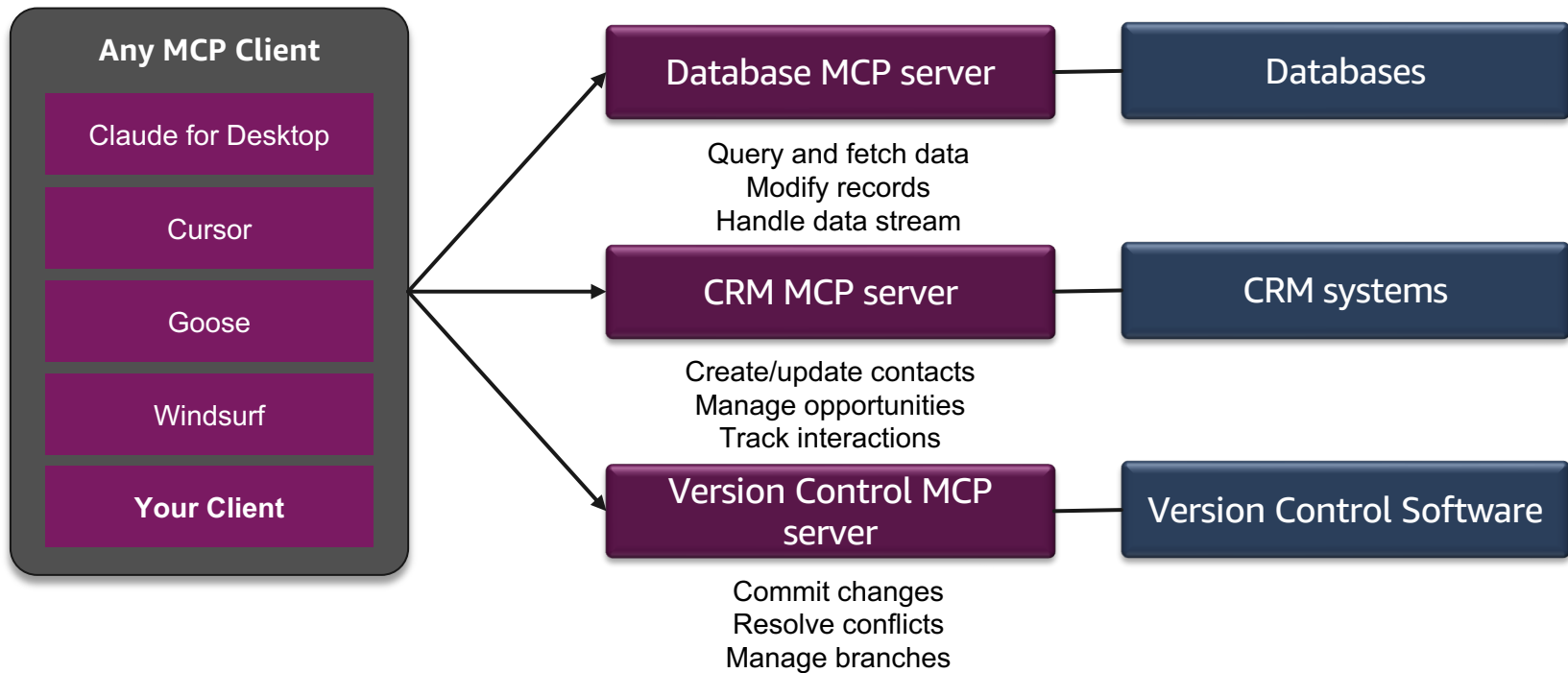
MCP: $N+M$ Connections



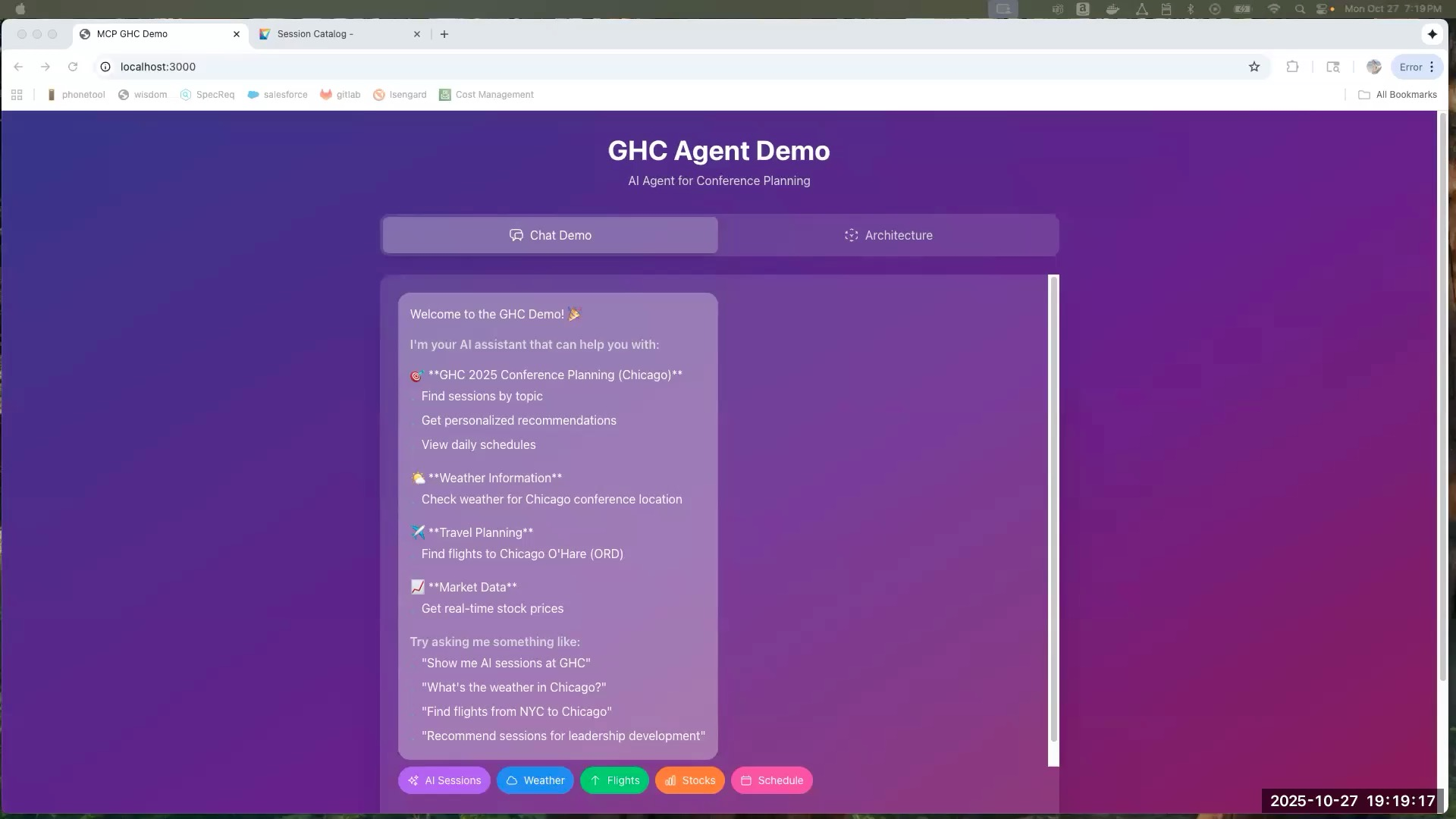
Models and data sources only need
to integrate once with MCP

6 Total Connections

With MCP: Standardized AI Development



Demo



GHC Agent Demo

AI Agent for Conference Planning

Chat Demo

Architecture

Welcome to the GHC Demo! 🎉

I'm your AI assistant that can help you with:

🎯 ****GHC 2025 Conference Planning (Chicago)****

Find sessions by topic

Get personalized recommendations

View daily schedules

🌤️ ****Weather Information****

Check weather for Chicago conference location

✈️ ****Travel Planning****

Find flights to Chicago O'Hare (ORD)

📈 ****Market Data****

Get real-time stock prices

Try asking me something like:

"Show me AI sessions at GHC"

"What's the weather in Chicago?"

"Find flights from NYC to Chicago"

"Recommend sessions for leadership development"

AI Sessions

Weather

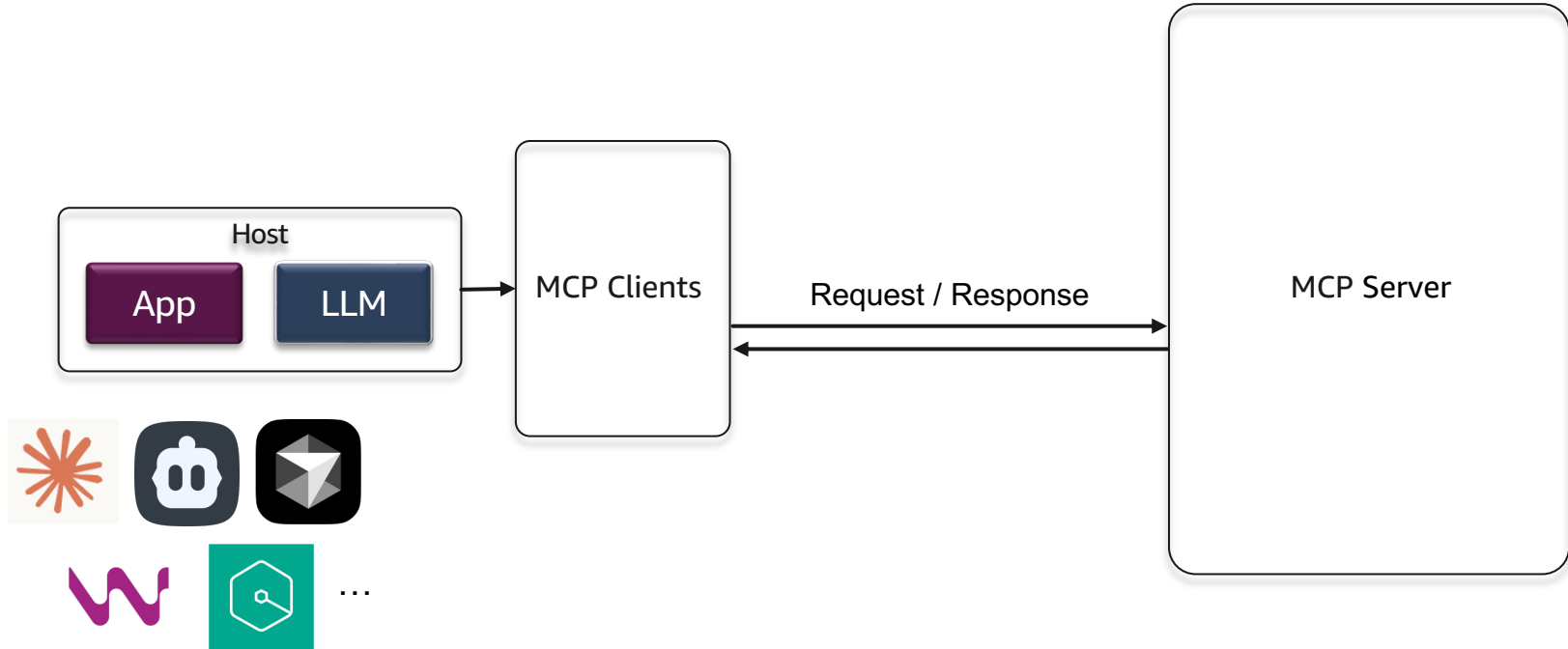
Flights

Stocks

Schedule

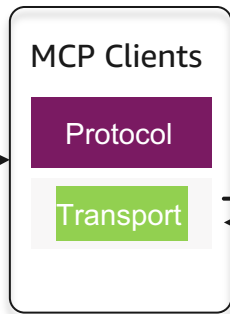
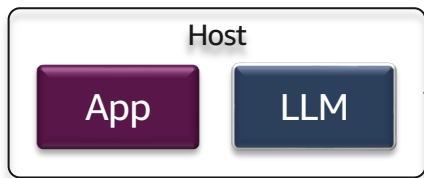
2025-10-27 19:19:17

MCP Core Components



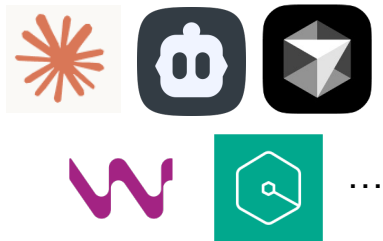
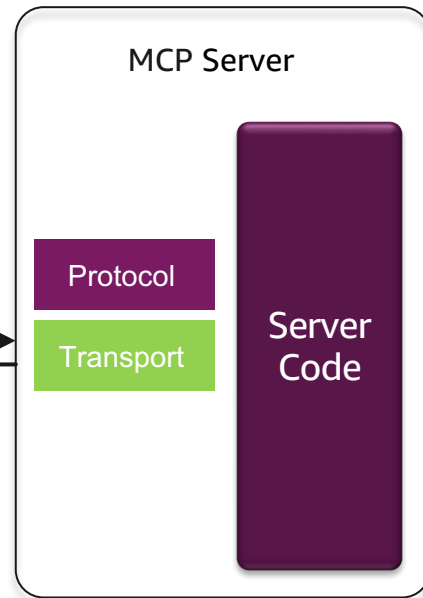
MCP Core Components - Protocol

```
await client.call_tool(  
    "get_weather",  
    {"location": "Chicago"}  
)
```

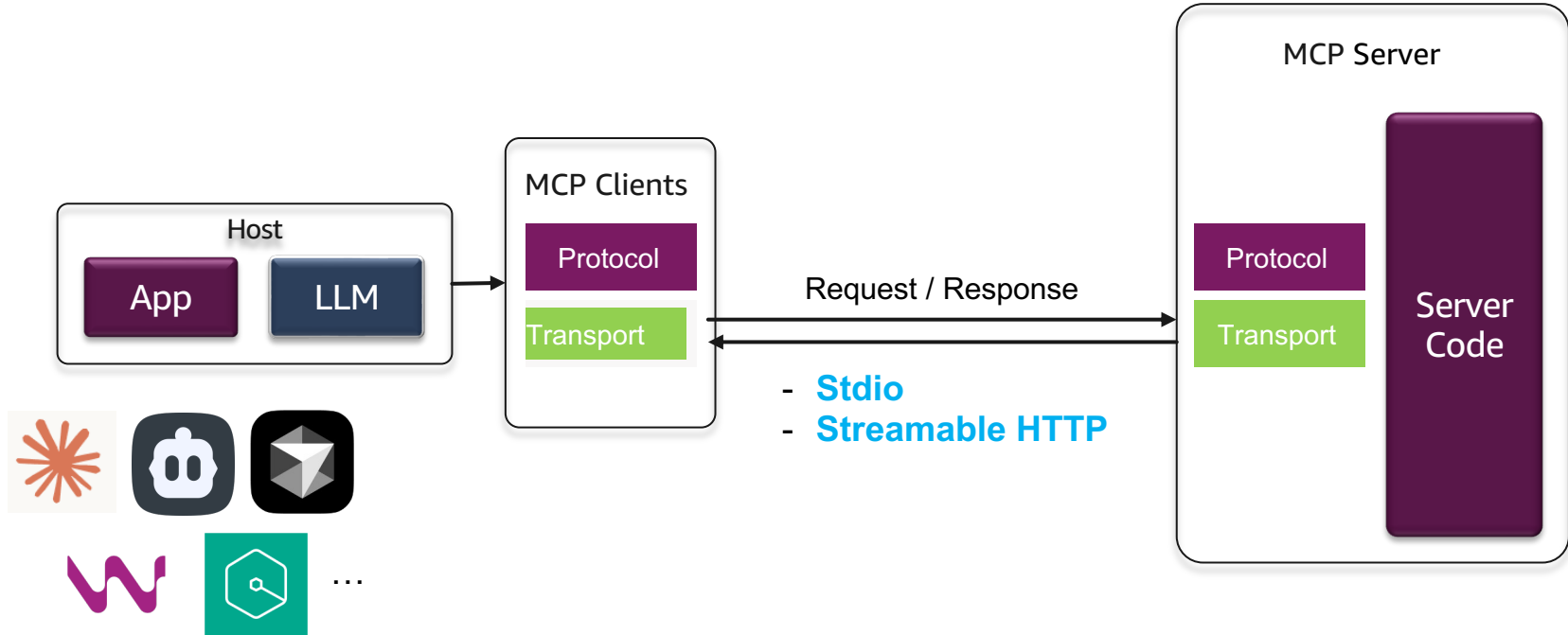


```
{  
  "jsonrpc": "2.0",  
  "id": "req-123",  
  "method": "tools/call",  
  "params": {  
    "name": "get_weather",  
    "arguments": {"location":  
      "Chicago"}  
  }  
}
```

Request / Response



MCP Core Components - Transport



MCP Core Components - Transport

MCP provides built-in transport implementations to suit different use cases

Stdio

- Communication through stdin/stdout streams
- Ideal for [local integrations](#)
- Simple implementation for CLI tools (or for shell script)

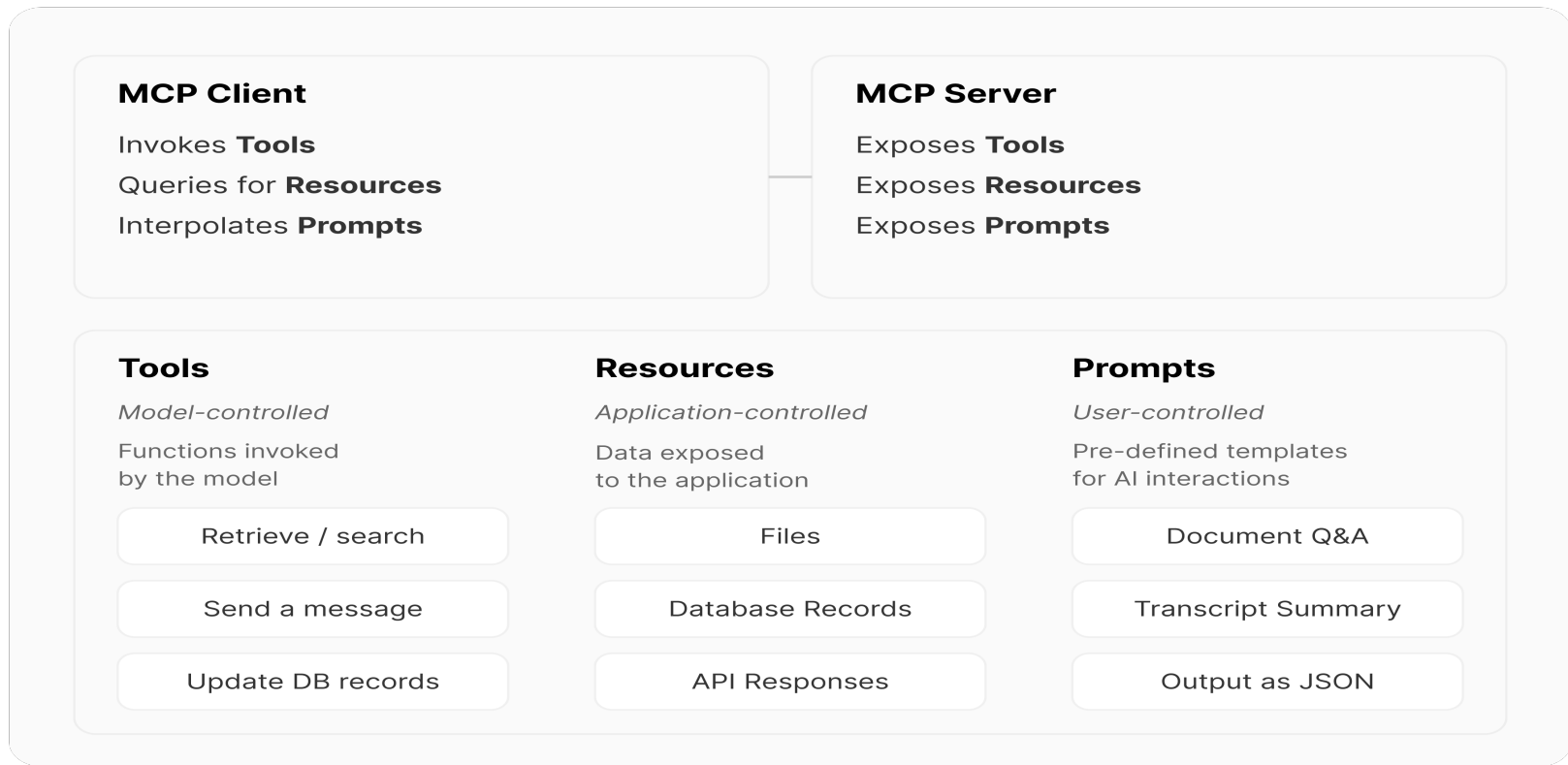
Streamable HTTP

- HTTP POST for client-to-server communication
- Support both [stateful](#) and [stateless sessions](#)

SSE (deprecated)

- [Stateful HTTP Connection](#)
- Backward compatibility

MCP Core Components – Tools, Resources and Prompts



End-to-end Components



Requester

- Request Generation
- Plug & Play
- Authentication

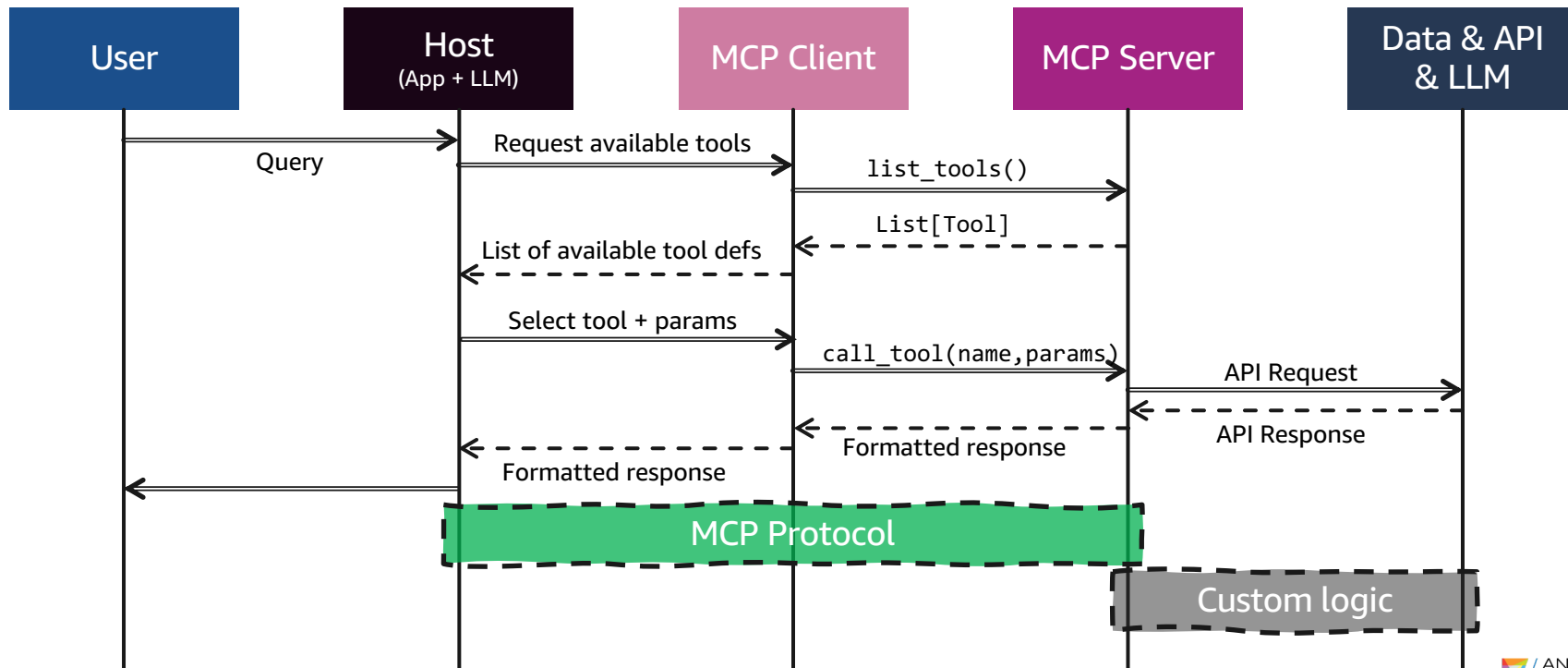
Agentic LLM Application

- Autonomous Agent
- Workflow
- Tool Calling

Context Server

- Database
- Internal/External API
- Agent Application

Request-Response Flow



MCP Server Implementation

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("weather")

@mcp.tool()
async def get_forecast(latitude: float, longitude: float) -> str:
    """Get weather forecast for a location.
    Args:
        latitude: Latitude of the location
        longitude: Longitude of the location
    """
    # Code implementation
    return f"Forecast for ({latitude}, {longitude}): Sunny, 72°F"

if __name__ == "__main__":
    mcp.run(transport='stdio')
```

1.Tool definition

2.Starting a
server

MCP Client Implementation

```
from mcp import ClientSession
from mcp.client.stdio import stdio_client
```

```
# Connect to server
```

```
stdio, write = await stdio_client({"command": "python", "args": [server_path]})
session = ClientSession(stdio, write)
await session.initialize()
```

1. Initialization

```
# Discover tools
```

```
tools = await session.list_tools()
print(f"Available tools: {[t.name for t in tools.tools]}")
```

2. Tool Discovery

```
# Example: select and call a tool
```

```
selected_tool = "get_weather" # Simplified tool selection
tool_args = {"location": "San Francisco"}
```

3. LLM Integration

```
# Execute tool call
```

```
result = await session.call_tool(selected_tool, tool_args)
```

4. Tool Invocation

```
# Clean up
```

```
await session.aclose()
```

Confidential & Proprietary - For internal use only.

Building MCP Clients

Strands Framework

Models

+

Agent

+

Tools

- Model Agnostic
- Tool Types
 - MCP Tools
 - Built-in Tools
 - Python Tools

```
from strands import Agent
from strands.models import
BedrockModel
```

```
bedrock_model = BedrockModel(
    model_id="us.amazon
.nova-premier-v1:0",
    temperature=0.3,
    top_p=0.8
)
```

```
# MCP Tools
```

```
from mcp import StdioServerParameters, stdio_client
from strands.tools.mcp.mcp_client import MCPClient
```

```
cost_mcp_server = MCPClient(
    lambda: stdio_client(
        StdioServerParameters(
            command="uvx",
            args=["awslabs.cost-explorer-mcp-server@latest"],
            env=env,
        )
    )
)
```

```
# Built-in Tools
```

```
from strands_tools import file_read, file_write, editor
```

```
tools = [file_read, file_write, editor]
```

```
# Python tools
```

```
from strands import tool
```

```
@tool
```

```
def weather_forecast(city: str, days: int = 3) -> str:
```

```
    """Get weather forecast for a city.
```

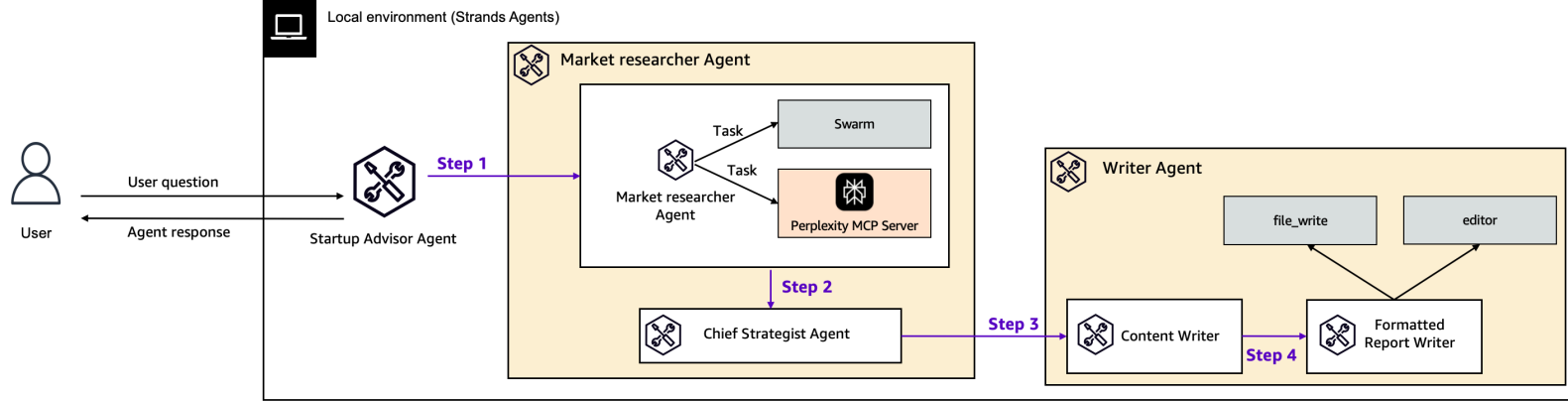
```
    Args:
```

```
        city: The name of the city
```

```
        days: Number of days for the forecast
```

```
    """
```

```
    # Implementation
```



Built-in tools

```

from dotenv import load_dotenv
from mcp import StdioServerParameters, stdio_client
from strands import Agent
from strands.tools.mcp import MCPClient
from strands_tools import swarm
  
```

```
load_dotenv()
```

```
PERPLEXITY_API_KEY = os.getenv("PERPLEXITY_API_KEY")
```

```
def market_research_team(project_description: str):
    response = str()
```

```

    try:
        perplexity_mcp_server = MCPClient(
            lambda: stdio_client(
                StdioServerParameters(
                    command="docker",
                    args=[
                        "run",
                        "-i",
                        "--rm",
                        "-e",
                        "PERPLEXITY_API_KEY",
                        "mcp/perplexity-ask",
                    ],
                    env={"PERPLEXITY_API_KEY": PERPLEXITY_API_KEY},
                )
            )
        )
    
```

```

    with perplexity_mcp_server:
        # Initialize Strands Agent with agent_graph
        tools = perplexity_mcp_server.list_tools_sync()
        lead_market_analyst = Agent(
            system_prompt="""Conduct thorough analysis of the products and competitors, providing detailed
            insights to guide marketing strategies. As the Lead Market Analyst at a digital marketing firm, you specialize
            in understanding markets for new products and services. You can do web search to gather more information.
            """,
            tools=tools + [swarm],
        )
  
```

LangGraph Framework

Use MCP tools

The `langchain-mcp-adapters` package enables agents to use tools defined across one or more MCP servers.

In an agent

In a workflow

Agent using tools defined on MCP servers

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent

client = MultiServerMCPClient(
    {
        "math": {
            "command": "python",
            # Replace with absolute path to your math_server.py file
            "args": ["/path/to/math_server.py"],
            "transport": "stdio",
        },
        "weather": {
            # Ensure you start your weather server on port 8000
            "url": "http://localhost:8000/mcp",
            "transport": "streamable_http",
        }
    }
)
tools = await client.get_tools()
agent = create_react_agent(
    "anthropic:claude-3-7-sonnet-latest",
    tools
)
math_response = await agent.ainvoke(
    {"messages": [{"role": "user", "content": "what's (3 + 5) x 12?"}]}
)
weather_response = await agent.ainvoke(
    {"messages": [{"role": "user", "content": "what is the weather in nyc?"}]}
```

CrewAI Framework

```
from crewai import Agent
from crewai_tools import MCPServerAdapter
from mcp import StdioServerParameters # For Stdio Server
```

Streamable HTTP Server:

```
server_params = {
    "url": "http://localhost:8001/mcp",
    "transport": "streamable-http"
}
```

Example usage (uncomment and adapt once server_params is set):

```
with MCPServerAdapter(server_params) as mcp_tools:
    print(f"Available tools: {[tool.name for tool in mcp_tools]}")
```

```
my_agent = Agent(
    role="MCP Tool User",
    goal="Utilize tools from an MCP server.",
    backstory="I can connect to MCP servers and use their tools.",
    tools=mcp_tools, # Pass the loaded tools to your agent
    reasoning=True,
    verbose=True
)
# ... rest of your crew setup ...
```

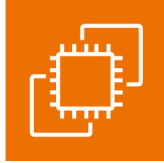
Deploy Anywhere



Lambda



ECS



EC2



EKS



Amazon Bedrock AgentCore

LLM based applications that support MCP and serve as MCP Clients



Amazon Q Dev CLI

Kiro

Claude Desktop

Cursor AI

Cline

modelcontextprotocol.io/clients

wisdomSpecReqsalesforcegitlabIsengardCost Management

Model Context ProtocolVersion 2025-06-18 (latest)

Search...

BlogGitHub

Examples

Example Servers

Example Clients

Tutorials

FAQs

Protocol

Specification

Key Changes

Architecture

Base Protocol

Client Features

Server Features

Schema Reference

Community

Governance and Stewardship

SEP Guidelines

Development

Versioning

Roadmap

Contributing

Examples

Example Clients

A list of applications that support MCP integrations

This page provides an overview of applications that support the Model Context Protocol (MCP). Each client may support different MCP features, allowing for varying levels of integration with MCP servers.

Feature support matrix

Client	Resources	Prompts	Tools	Discovery	Sampling	Roots	Elicitation	
Sire	✗	✗	✓	?	✗	✗	?	Support:
AgentAI	✗	✗	✓	?	✗	✗	?	Agent Li written i tools su
AgentiFlow	✓	✓	✓	✓	✗	✗	?	Support: prompts resource code AI multi-ag workflo
AIQL TUUI	✓	✓	✓	✓	✓	✗	?	Support: prompts and sam MCP ser Multi-LL
Amazon Q CLI	✗	✓	✓	?	✗	✗	?	Support: and tool:
Amazon Q IDE	✗	✗	✓	✗	✗	✗	?	Support:

On this page

Feature support matrix

Client details

Sire

AgentAI

AgentiFlow

AIQL TUUI

Amazon Q CLI

Amazon Q IDE

Apify MCP Tester

Augment Code

BeeAI Framework

BoltAI

ChatWise

Claude Code

Claude.ai

Claude Desktop App

Chorus

Cline

CodeGPT

Continue

Copilot-MCP

Cursor

Daydreams

Emacs MCP

Building MCP Servers

MCP Server

Location

Latency

Accessibility

Security

Use Case

Debugging

Setup Complexity

Performance Testing

Local MCP Server

Runs on the developer's own machine (localhost).

Low latency, fast feedback loop for testing/debugging.

Only accessible from the local machine (unless configured).

Generally safer for early development; limited attack surface.

Ideal for development, prototyping, and testing.

Easier to attach debuggers and inspect logs.

Simple to run with local dependencies.

Limited by local machine's resources.

Remote MCP Server

Hosted on a different machine, server, or cloud environment.

Higher latency due to network communication.

Can be accessed by multiple clients over a network.

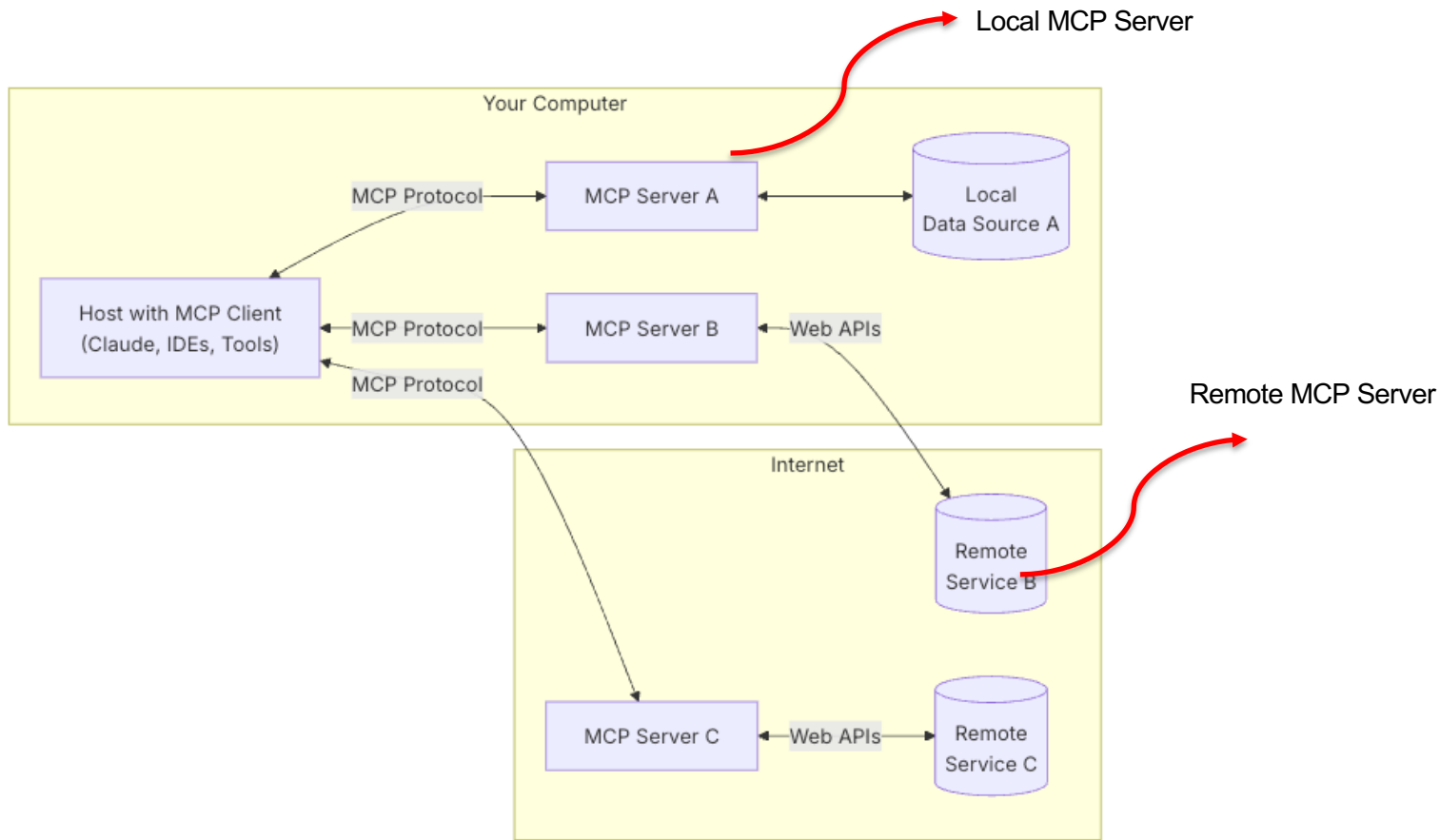
Needs security measures (auth, encryption, etc.) for safe access.

Used for staging, production, or collaborative environments.

May require remote logging and monitoring tools.

Needs deployment pipeline and network configuration.

Allows testing under real-world, multi-user load conditions.



MCP Architecture Evolution

- MCP protocol is evolving to be more cloud-native
 - transition from simple local deployments to distributed cloud-native patterns
 - enable greater scalability and flexibility in how AI agents interact with tools.

[Local Tools]



Agent & Tools

[Remote Servers]



Agent
(Client)

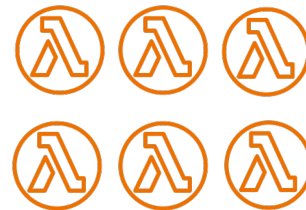


Tools
(Servers)

[Server Farm]



Agent
(Client)



Server Farm

Running MCP Server on Serverless

- Maintenance-free
- Faster Scaling
- Security
- CI/CD

1. Stateless



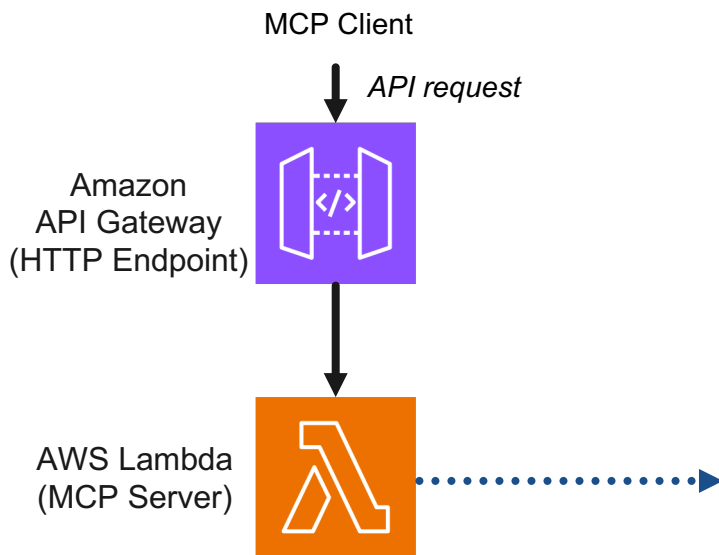
AWS Lambda

2. Stateful



AWS Fargate

Stateless MCP Server



```
from awslabs.mcp_lambda_handler import MCPLambdaHandler

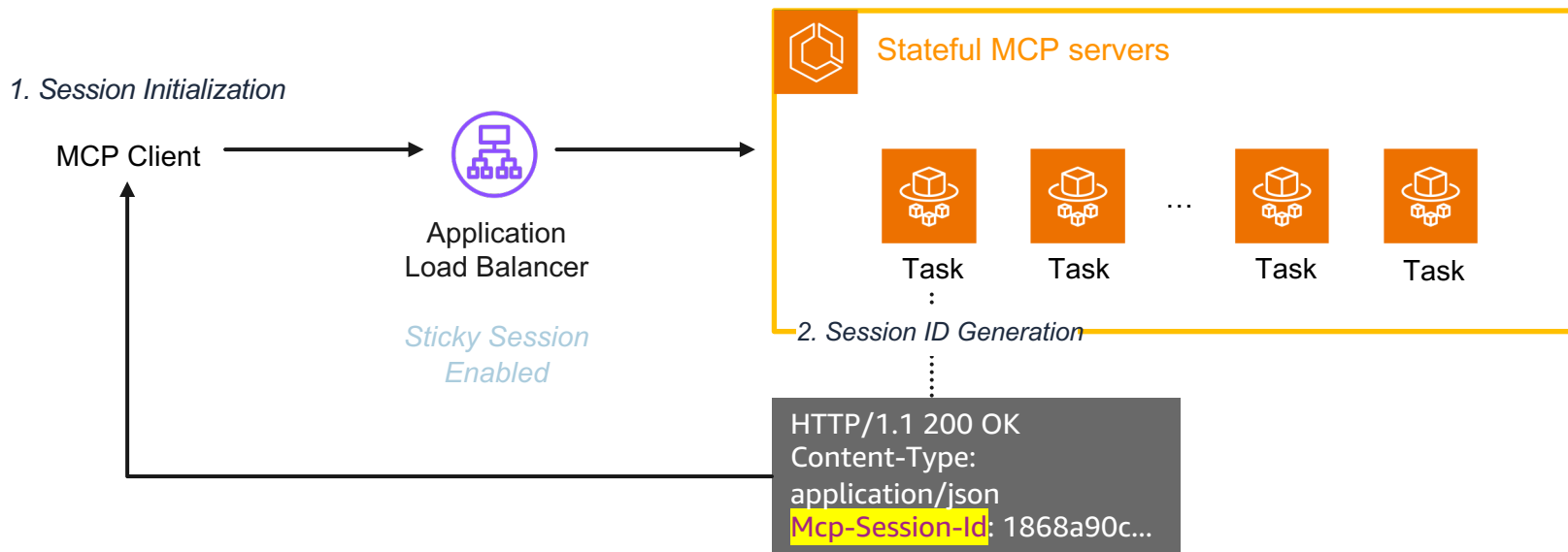
mcp = MCPLambdaHandler(name="my-mcp-server", version="1.0.0")

@mcp.tool()
def calculate(a: int, b: int, operation: str) -> dict:
    result = None
    if operation == "add":
        result = a + b
    elif operation == "subtract":
        result = a - b
    elif operation == "multiply":
        result = a * b
    elif operation == "divide":
        result = a / b if b != 0 else "Error: Division by zero"
    return {"result": result}

def lambda_handler(event, context):
    return mcp.handle_request(event, context)
```

Stateful MCP Server

- MCP servers generate a session ID during initialization that clients must include in all subsequent request headers.



Stateful MCP Server

- MCP servers generate a session ID during initialization that clients must include in all subsequent request headers.

1. Request with Session ID

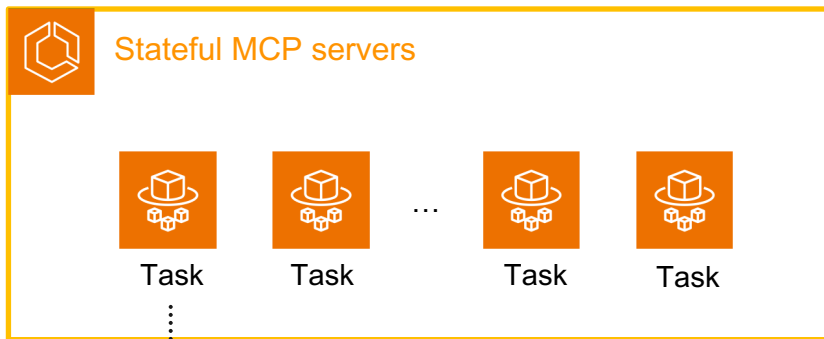
MCP Client



Application
Load Balancer

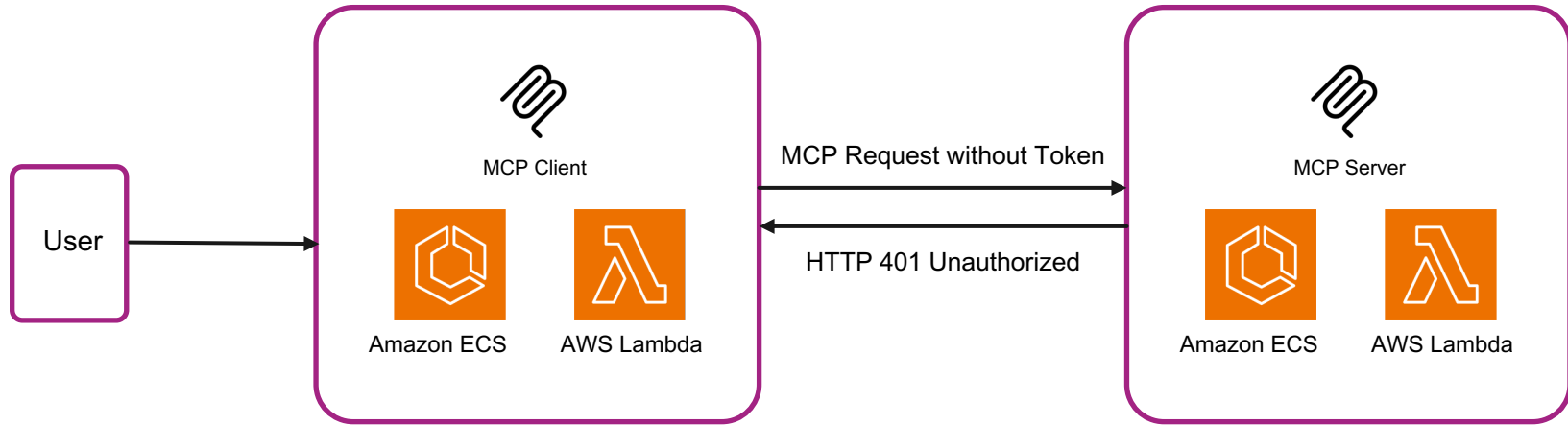
*Sticky Session
Enabled*

```
POST /mcp HTTP/1.1
Content-Type:
application/json
Mcp-Session-Id: 1868a90c...
```

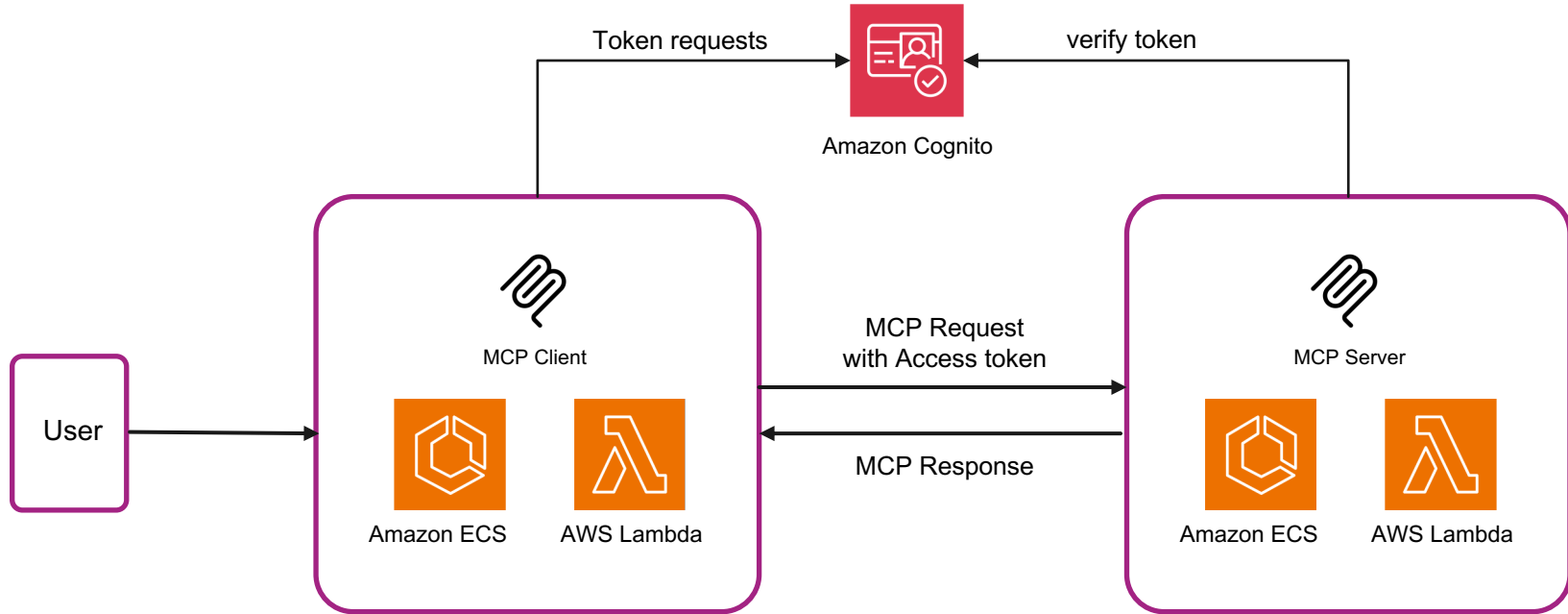


2. Stateful Tool Execution

Authorization Flow

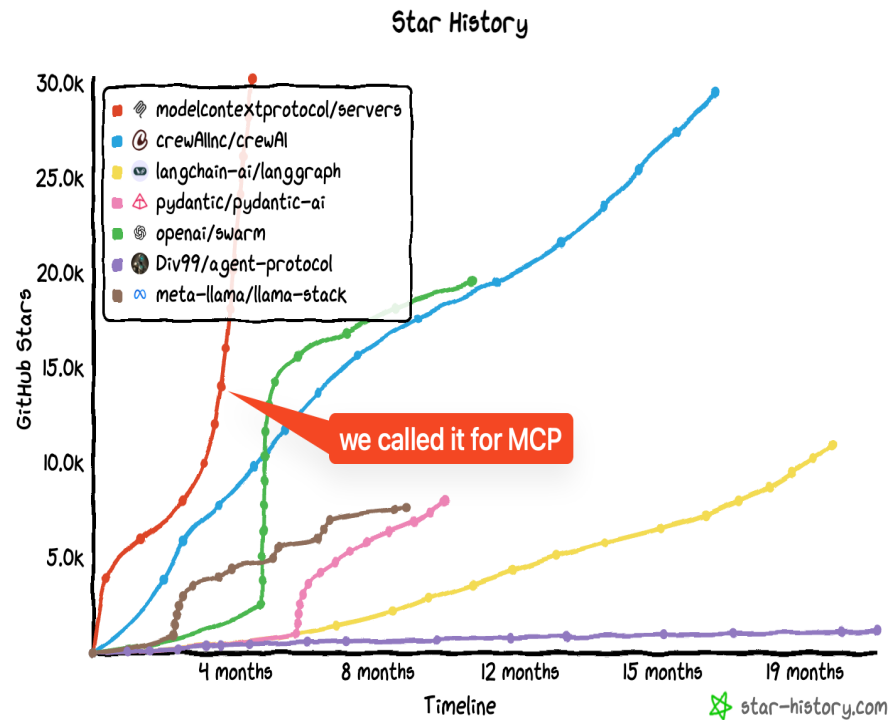
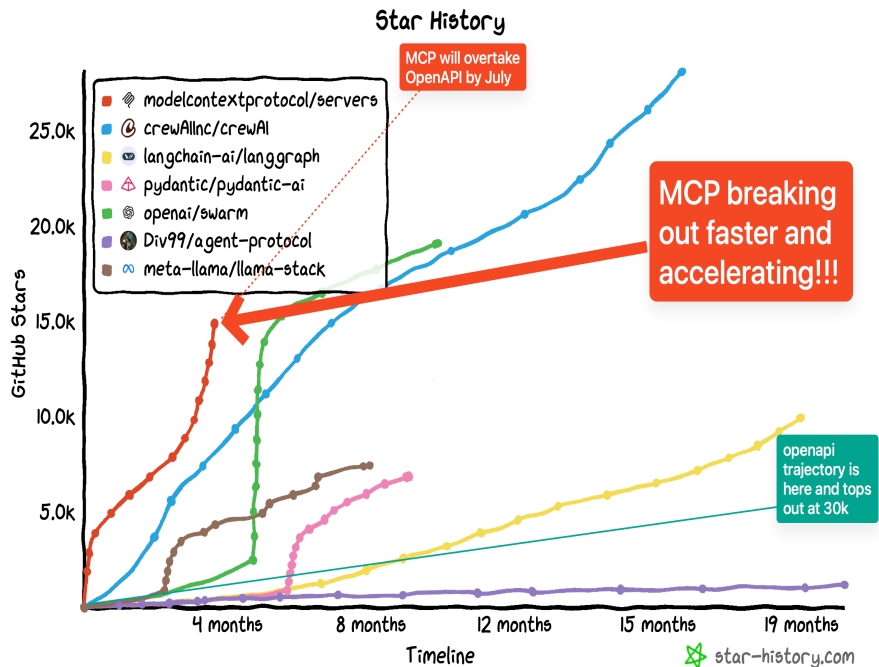


Authorization Flow



Industry Trends

MCP popularity



What is the industry doing ?



API Tracker / Expedia / Servers /



company

Expedia MCP servers

Overview APIs SDKs MCPs Integrations Specs Technologies Alternatives Endpoints

Products Industries Customers Learning Support Company Salesforce+

Developers Home Blog Documentation APIs Discover Build Connect

Salesforce Developers Blog

Introducing MCP Support Across Salesforce

MOHITH SHRIVASTAVA

servicenow Community

Product Hubs Connect Blogs Resources ServiceNow University

ServiceNow Community Discussions Developer Developer articles How to create your own ServiceNow MCP Server

How to create your own ServiceNow MCP Server

VaranAwesomenow
Mega Sage

3 weeks ago · edited
3 weeks ago

How to create your own ServiceNow MCP Server

Code: [anilvaranasi/ServiceNowMCPServer](#)

In order to implement Agentic AI one of the first steps it to be able to build AI agents that can interact with ServiceNow. One of the easiest ways to interact with ServiceNow is via REST APIs. This blog covers steps on how to build an MCP server that helps in interacting with ServiceNow via REST APIs and consuming the MCP Server via Claude desktop MCP client.

This reader built knowledge of various not understanding of MCP (Model Context Protocol). To not needed built



Build Learn Community Blog

Search Slack Developer

Share



Introducing Slack MCP Support, Real-time Search API, and Updates to Terms of Service

Today, we're unveiling new capabilities that redefine how developers, customers, and partners securely interact with Slack data. At the heart of this evolution are two key innovations: support for the Model Context Protocol (MCP) and the launch of our Real-time Search API. Together, these advancements lay the foundation for building intelligent, contextual AI experiences, without compromising on customer trust or our commitment to interoperability.

Confidential & Proprietary



Products Solutions Developers Why Twilio Pricing

Contact sales Start for free

Twilio Blog Overview Developers Industry Insights Product News Events Partners Company

Use the Twilio Alpha MCP (Model Context Protocol) Server to Automate Development

Workshop



Thank you

Connect with us



Neelam Koshiya

Principal - GenAI @ Amazon



Nafi Diallo

AWS Automated Reasoning / AI



Resources



AWS Solutions Architect - Associate Certification Guide

AWS is the global leader in cloud computing, and the AWS Certified Solutions Architect - Associate (SAA-C03) credential is the benchmark for validating your ability to design resilient, high-performing, and cost-optimized cloud solutions. This guide is your complete pathway from certification preparation to professional mastery in cloud architecture. This comprehensive resource tackles the SAA-C03 exam while building real-world skills that matter in today's cloud-first business environment.

This book bridges the critical gap between theoretical knowledge and practical application. You will master core AWS services, architectural patterns, and design principles through a structured learning approach that emphasizes hands-on experience. The book combines rigorous exam preparation with business-focused scenarios, ensuring you are not just ready to pass the certification but equipped to design, develop, deploy, and manage robust cloud solutions. From cost optimization strategies to migration planning, security best practices to scalability patterns, every concept is reinforced with practical examples and real-world case studies.

By the end of this book, you will confidently design scalable AWS architectures, pass the AWS Solutions Architect - Associate (SAA-C03) certification exam, and implement cost-effective cloud solutions. You will master migration strategies, security best practices, and infrastructure as code deployment techniques for real business scenarios.

WHAT YOU WILL LEARN

- Design resilient AWS architectures using EC2, S3, RDS, and VPC to meet business requirements and handle varying workloads.
- Implement security best practices, including IAM policies, encryption, network security, and compliance frameworks, to protect resources.
- Optimize cost and performance through right sizing, auto scaling, storage optimization, and monitoring solutions.
- Master disaster recovery using Multi-AZ deployments, cross-region replication, and automated backups for business continuity.
- Apply real-world scenarios and hands-on labs that mirror exam questions and prepare for professional architecture challenges.

WHO THIS BOOK IS FOR

This book is ideal for current and aspiring IT professionals, cloud engineers, and developers seeking the AWS Solutions Architect - Associate certification. It is for both beginners entering AWS and intermediate users advancing their cloud architecture skills. Readers should have a basic understanding of operating systems, networking, and cloud computing concepts. It is suitable for beginners entering AWS and intermediate users advancing their cloud architecture skills.



BPB PUBLICATIONS
www.bpbonline.com

AWS Solutions Architect - Associate Certification Guide

Neelam Koshiya
Jing Ning



AWS Solutions Architect Associate Certification Guide

A comprehensive guide from AWS Solutions Architect - Associate certification to real-world applications



Neelam Koshiya
Jing Ning

